
Project 4: DNS Resolution

1 Overview

In this lab project, you will implement two types of DNS resolvers: (1) a **recursive resolver** and (2) an **iterative resolver**.

Both resolvers take a domain name as input and return the corresponding IPv4 address, if one exists. You will interact with a provided **fake DNS hierarchy** that simulates root, TLD, and authoritative servers.

The goal of this project is to understand how DNS resolution works at the protocol level by constructing and parsing DNS messages directly.

2 What You Need to Implement

You will implement the following two functions:

```
def recursive_resolve(name: str, recursive_server: str) -> str | None:
```

```
def iterative_resolve(name: str, root_server: str) -> str | None:
```

Both functions should return an IPv4 address as a string if resolution succeeds, and **None** otherwise.

3 Recursive vs. Iterative Resolution

3.1 Recursive Resolver

For this project, the recursive resolver must send a single DNS query to a recursive DNS server.

Your responsibility is to:

- Construct a valid DNS query packet with recursion enabled
- Send it to the given server
- Parse the response and extract the final answer

The recursive server performs all resolution on your behalf. Your implementation does **not** need to follow referrals or contact multiple servers.

3.2 Iterative Resolver

The iterative resolver performs resolution manually by walking the DNS hierarchy.

Starting from the root server, your resolver must:

1. Send a query to the root server
2. Inspect the response
3. Determine the next server to contact
4. Repeat until an answer is found or resolution fails

This means your implementation must interpret DNS responses and follow referrals step-by-step.

4 DNS Protocol Scope and Simplifications

You are not expected to implement the full DNS protocol. This project uses a simplified subset designed to focus on core resolution behavior.

Your implementation should be able to construct DNS query messages, parse responses, and extract information from the following record types:

- A
- NS
- CNAME

The fake DNS system follows these assumptions:

- Each query contains exactly one question
- Responses include only A, NS, and CNAME records
- The additional section will always include the IP address of the next server
- Domain names appear in full (no compression)
- All communication uses UDP

You do not need to handle advanced DNS features such as IPv6 or DNSSEC.

You should use the provided networking helper utilities for sending and receiving UDP packets.

4.1 Handling CNAME Records

If a response contains a CNAME record instead of an A record, the domain is an alias.

Your resolver should extract the canonical name from the CNAME record and restart resolution using that name. For the recursive resolver, this means issuing a new query to the recursive server using the canonical name. For the iterative resolver, this means restarting resolution from the root server using the canonical name.

You may assume that CNAME chains are short, and that a valid A record will eventually be reachable.

5 Testing and Submitting Your Code

5.1 Testing

To test your code, follow these steps *in order*:

1. Start the fake DNS servers in one terminal:

```
python3 run_fake_dns.py
```

2. Test your resolvers on a set of predefined domains.

```
python3 student_tester.py
```

For testing a single domain, use:

```
python3 dns_lookup.py [domain]
```

This will print the result from your resolver and is useful for debugging. A list of available domains is available in `tests/domains.txt`.

5.2 Submission

You will submit two files, `iterative_resolver.py` and `recursive_resolver.py` to Gradescope.

6 Helpful Resources

You will need to construct and parse DNS packets manually. The following references may be useful:

- The DNS message format is defined in RFC 1035, found at <https://datatracker.ietf.org/doc/html/rfc1035>. You do not need the entire document, just focus on the header, question, and answer format.
- For Python byte manipulation, see <https://docs.python.org/3/library/struct.html>.